

 Sign in to Medium with Google ✕



Sandipan Dey
sandipan.dey@gmail.com



Sandipan Dey
geekforeverumbc@gmail.com



Bhuvana Kundumani
Sep 17, 2019 · 5 min read · [Listen](#)



Simple Neural Network with BCELoss for Binary classification for a custom Dataset

$$l_n = -w_n [y_n \cdot \log x_n + (1 - y_n) \cdot \log(1 - x_n)]$$

In our previous blog [Writing a Dataloader for a custom Dataset \(Neural network\) in Pytorch](#), we saw how to write custom Dataloaders for a dataset. We converted the [Boston Dataset](#) into a classification problem, made data batches (DataLoaders) that are ready to be fed into the pytorch neural network architecture. In this blog, we will be focussing on how to use BCELoss for a simple neural network in Pytorch.

Our dataset after preprocessing has 12 features and 1 target variable. We will have a neural network with 1 hidden layer with 128 neurons.

```
# Simple Neural network
input_size = 12
hidden_size = 128
num_classes = 1
num_epochs = 5
learning_rate = 0.001
BATCH_SIZE_1 = 101 #train_loader as it has 404 observations
BATCH_SIZE_2 = 51 #test_loader as it has 102 observations
```

We are going to use BCELoss as the loss function. BCELoss creates a criterion that measures the Binary Cross Entropy between the target and the output. You can read more about BCELoss [here](#). If we use BCELoss function we need to have a sigmoid layer in our network. A sigmoid function is a type of activation function, and more specifically defined as a squashing function. Squashing functions limit the output to a range between 0 and 1.

```
# Neural Network should have a sigmoid activation function if you are
using BCELoss()
class LinearModel(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(LinearModel, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.fc2 = nn.Linear(hidden_size, num_classes)
        self.relu = nn.ReLU()

    def get_weights(self):
        return self.weight

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = F.sigmoid(self.fc2(out)) #sigmoid as we use BCELoss
        return out
```

Now we have our Neural Network architecture in place, let us see our training and testing functions.

TRAIN FUNCTION

Train function has the input parameters — model, device, train_loader and optimizer.

```
#TRAINING THE NETWORK
def train(model, device, train_loader, optimizer):
    model.train()
    y_true = []
    y_pred = []

    for i in train_loader:

        #LOADING THE DATA IN A BATCH
        data, target = i

        #MOVING THE TENSORS TO THE CONFIGURED DEVICE
        data, target = data.to(device), target.to(device)

        #FORWARD PASS
        output = model(data.float())
        loss = criterion(output, target.unsqueeze(1))

        #BACKWARD AND OPTIMIZE
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # PREDICTIONS
        pred = np.round(output.detach())
        target = np.round(target.detach())
        y_pred.extend(pred.tolist())
        y_true.extend(target.tolist())

    print("Accuracy on training set is" ,
          accuracy_score(y_true, y_pred))
```

model.train() tells the program that the model is in training mode. This becomes important when we have dropout layers in our network. By default, a PyTorch neural network model is in train() mode. As long as there's no dropout layer (or batch normalization) in the network, you don't need to worry about train() mode vs. eval() mode. However, I usually use it in my train function by default. We would be using model.eval() in the test function.

We load the data, target from the train_loader and then create the model using output = model(data.float())

Since the batch size for the train_loader is 101, data and target would be of shapes -torch.Size([101, 12]) torch.Size([101]) respectively. output is of shape torch.Size([101, 1]). So when we are calculating our loss, we have to use target.unsqueeze(1) to change



Bhuvana Kundumani
128 Followers

Data Scientist (NLP) @ Shell

Follow

More from Medium

Gabriele Mattioli in MLearning.ai

CIFAR10 image classification in PyTorch



Marco Sangalli in Towards Data Science

Implementing Custom Loss Functions in PyTorch



Alessandro Lamberti in Artificial Intelligence

Multi-Task Deep Learning with Pytorch



Zain Baqai in Towards Data Science

Time Series Forecasting with Deep Learning in PyTorch (LSTM-RNN)




```
loss = criterion(output, target.unsqueeze(1))
```

If we do not use `unsqueeze`, we will get the following error- `ValueError`:

```
Target size (torch.Size([101])) must be the same as input size
(torch.Size([101, 1]))
```

We have calculated the loss function. Then we back propagate and optimize using the commands below.

```
#BACKWARD AND OPTIMIZE
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

We have to make predictions on the training dataset to calculate the training set accuracy. We are using accuracy as the performance metric here. Please note that our dataset has a class imbalance (In simpler terms, we have 422 observations for class — 0 and 84 observations for class — 1) and hence this is nto the besty metric for this case. I would be posting a detailed blogpost on the performance metric.

```
# PREDICTIONS
pred = np.round(output.detach().numpy())
target = target.float()
y_true.extend(target.tolist())
y_pred.extend(pred.reshape(-1).tolist())
```

While making predictions, we need to use `output.detach().numpy()` ,

Otherwise we get the following error - *RuntimeError: Can't call numpy() on Variable that requires grad. Use var.detach().numpy() instead.*

Since we have a sigmoid layer at the last layer our outputs are in the range 0 to 1. We use the round function in Python to round off these values to either 0.0 or 1.0.

In `y_pred.extend(pred.reshape(-1).tolist())` , `reshape(-1)` is used to convert the size of `pred` from (101,1) to (101,) `tolist()` converts the array to a list and `extend` is used to append the list to `y_pred`.

Similarly in `y_true.extend(target.tolist())`, `reshape(-1)` is not needed here as we already have `target` in shape [101]. `tolist()` converts the torch tensor to a list and `extend` is use to append the list to `y_true`.

TEST FUNCTION

Test function has the input parameters — `model`, `device` and `test_loader`.

```
#TESTING THE MODEL
def test(model, device, test_loader):
    #model in eval mode skips Dropout etc
    model.eval()
    y_true = []
    y_pred = []

    # set the requires_grad flag to false as we are in the test mode
    with torch.no_grad():
        for i in test_loader:

            #LOAD THE DATA IN A BATCH
            data,target = i

            # moving the tensors to the configured device
            data, target = data.to(device), target.to(device)

            # the model on the data
            output = model(data.float())

            #PREDICTIONS
            pred = np.round(output)
            target = target.float()
            y_true.extend(target.tolist())
            y_pred.extend(pred.reshape(-1).tolist())

    print("Accuracy on test set is" , accuracy_score(y_true,y_pred))
    print("*****")
```

with torch.no_grad() deactivates autograd. Since in test mode, we wouldn't be using backprop . This also helps in reducing memory usage and speed up computations.

We load the data and target from the `test_loader`. Create model on data using `output = model(data.float())`

We have to make predictions on the test dataset to calculate the accuracy on test dataset.

```
#PREDICTIONS
pred = np.round(output)
target = target.float()
y_true.extend(target.tolist())
y_pred.extend(pred.reshape(-1).tolist())
```

Please note that here in the test function we are not using

`output.detach().numpy()` as we are in the eval mode and we have used *with torch.no_grad()* in our test function.

CREATING MODEL AND SETTING UP LOSS AND OPTIMIZER:

The following commands create a model, sets up the loss to BCELoss and Adam optimizer is used.

```
# Creating model and setting loss and optimizer.
model = LinearModel(input_size, hidden_size, num_classes).to(device)
```



```
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

Let us call the train and test function for 5 epochs and see how our model behaves.

```
for epoch in range(num_epochs):
    train(model,device,train_loader,optimizer)
    test(model,device,test_loader)
```

The output when the above commands were executed are given below.

```
Accuracy on training set is 0.7326732673267327
Accuracy on test set is 0.7941176470588235
*****
Accuracy on training set is 0.8193069306930693
Accuracy on test set is 0.7647058823529411
*****
Accuracy on training set is 0.7425742574257426
Accuracy on test set is 0.7941176470588235
*****
Accuracy on training set is 0.8267326732673267
Accuracy on test set is 0.7941176470588235
*****
Accuracy on training set is 0.8514851485148515
Accuracy on test set is 0.7941176470588235
*****
```

Code is available at github repo —
<https://github.com/bhuvanakundumani/BCEvsBCEWithLogitsloss.git>

Sign up for Analytics Vidhya News Bytes

By Analytics Vidhya

Latest news from Analytics Vidhya on our Hackathons and some of our best articles! [Take a look.](#)

Your email

 Get this newsletter

By signing up, you will create a Medium account if you don't already have one. Review our [Privacy Policy](#) for more information about our privacy practices.